

Java - Streams



Dozent: Prof. Dr. Michael Eichberg
Kontakt: michael.eichberg@dhbw.de, Raum 149B
Version: 2.0 [Themed]

Teile der Folien sind inspiriert von: Th. Letschert - Funktionale Programmierung in Java.

Folien: <https://delors.github.io/prog-adv-java-streams/folien.de.rst.html>
<https://delors.github.io/prog-adv-java-streams/folien.de.rst.html.pdf>
Kontrollfragen: <https://delors.github.io/prog-adv-java-streams/kontrollfragen.de.rst.html>
Fehler melden: <https://github.com/Delors/delors.github.io/issues>

(Aktuelle) Herausforderungen

Verarbeitung von:

- großen Datenmengen
 - die nicht (ohne weiteres) in den Speicher passen oder
 - bei denen es keinen Sinn macht diese vorab vollständig in den Speicher zu laden, da immer nur ein kleiner Abschnitt analysiert werden muss
- Daten, die kontinuierlich verarbeitet werden müssen

Beispiel

Konkrete Szenarien

- HTTP Request/Response Handling (Audio/Video Streaming)
 - Verarbeitung von AI Streaming Responses
 - Ver-/Entschlüsselung von großen Datenmengen
 - Verarbeitung große Logdateien
 - Kontinuierliche Verarbeitung von Daten
 - Verarbeitung von Ereignissen (Event)
 - Echtzeitdatenverarbeitung von kontinuierlichen Daten von (IoT) Sensoren, Finanzdaten/-transaktionen
 - Aufbereitung großer Wörterbücher (z.B. für Passwortwiederherstellung)
- Parallele Verarbeitung und effiziente Nutzung von modernen CPU-Architekturen

Insbesondere die korrekte Nutzung ist häufig schwierig und hier ist die Verwendung von transparent parallelisierten Streams sehr hilfreich. (Z. B. mit Java Parallel Streams oder `.par` in Scala ist eine transparente Parallelisierung möglich.)

- Entwicklung von Software nahe am Domänenmodell, um die korrekte Implementierung zu unterstützen. Das Ziel sollte eine möglichst geringe Repräsentationslücke ( *Low representational gap*) zwischen Code und Domänenmodell sein.

▲ Achtung!

Im Folgenden betrachten wir die Java Stream API und nicht Java I/O Streams.

	Java I/O Streams	Java Stream API
Package	<code>java.io.*</code> / <code>java.nio.*</code>	<code>java.util.stream.*</code>
Fokus	Byte-/Zeichentransport	Datenverarbeitung
Paradigma	Imperativ	Funktional/Deklarativ

Imperativ Programmierung:

Das Vorgehen wird detailliert durch konkrete Anweisungen beschrieben, die genau vorgeben welche einzelnen Schritte von dem Computer ausgeführt werden sollen, um das Ziel zu erreichen.

Viele gängige *general-purpose* Programmiersprachen (C, C++, Rust, Java, Go, JavaScript, Python) sind im Kern imperative Programmiersprachen.

Deklarative Programmierung:

Das Ziel ist es auszudrücken *was* erreicht werden soll, ohne das *wie* genau anzugeben.

(Prominentes Beispiel für eine deklarative Programmiersprache: SQL als Datenbankabfragesprache.)

◆ Bemerkung

- Auch für gängige Programmiersprachen, die im Kern imperativ sind, ist zu beobachten, dass mehr und mehr Ideen und Konzepte aus der deklarativen Programmierung Einzug in diese Sprachen - insbesondere auch über APIs - finden.
- Es ist in der Zwischenzeit so, dass die Grenzen zwischen (klassischen) prozeduralen (d. h. primär imperativen) Programmiersprachen und funktionalen sowie deklarativen Programmiersprachen verschwimmen.
- Scala - als ein Beispiel für eine moderne Programmiersprache - ist von Grund auf als objektorientiert-funktionale Sprache entworfen wurde.

Streams - Einführung

Pragmatische Sicht:

Streams^[1] sind umgeformte Sammlungen, die durch die Umformung für funktional-orientierte Massen-Operationen geeignet sind.

Konzeptionelle Sicht:

Streams erlauben die „korrekte, lesbare (domänennahe)“ Verarbeitung von Daten mit Hilfe von Konzepten und Ideen aus der funktionalen und deklarativen Programmierung.

Beispiel

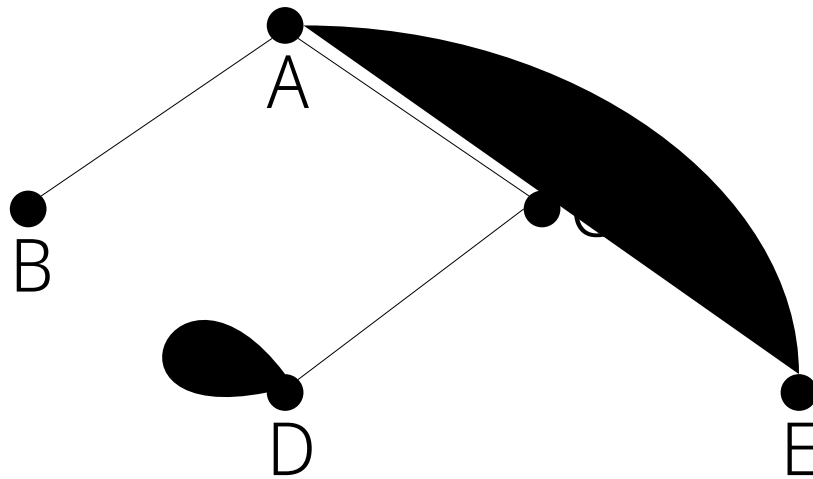
```
1 import java.util.Arrays;
2 import java.util.List;
3 import java.util.stream.Collectors;
4
5 List<Integer> l = Arrays.asList(1, 2, 3, 4, 5, 6, 7, 8, 9);
6 List<Integer> ll = l
7     .stream()                // list → stream
8     .filter(x → x % 2 == 0)  // filter list with predicate
9     .map(x → 10 * x)         // map each element to a new one
10    .collect(Collectors.toList()); // back to a list
11 ll.forEach(x → System.out.println(x));
```

Ausgabe: 20 40 60 80

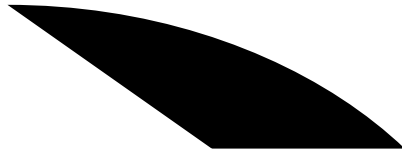
[1] Th. Letschert

Streams API - kurze Historie

- die Streams API wurde mit Java 1.8 eingeführt und wird seit dem ständig **weiterentwickelt**.



٤



Streams mit primitiven Daten und Objekten

- `Stream<T>` ist der Typ der Streams mit Objekten vom Typ `T`
- Streams mit primitiven Daten:

- `IntStream`
- `LongStream`
- `DoubleStream`

Dies Streams mit primitiven Daten arbeiten in vielen Fällen effizienter jedoch sind manche Operationen nur auf `Object`-Streams erlaubt. „Primitive“ Streams können mit der Methode `boxed` in `Object`-Streams umgewandelt werden.



Beispiel

```
1 IntStream isPrim = IntStream.range(1, 10);  
2 Stream<Integer> isObj = isPrim.boxed();
```

Erzeugung von Streams

Statische Methoden in `Arrays`

- Die Klasse `java.util.Arrays` hat mehrere überladene statische stream-Methoden, mit denen Arrays in Ströme umgewandelt werden können.
- Die Streams können Objekte oder primitive Daten enthalten.

Beispiel

```
1 // Stream of primitive data:
2 IntStream isP = Arrays.stream(new int[] { 1, 2, 3, 4, 5, 6, 7, 8, 9, 0 });
3 // Stream of objects:
4 Stream<Integer> isO = Arrays.stream(
5     new Integer[] { 1, 2, 3, 4, 5, 6, 7, 8, 9, 0 }
6 );
```

Statische Methoden in `Stream`

- Das Interface `java.util.stream.Stream` enthält mehrere statische Methoden mit denen Streams erzeugt werden können.
- Für die Klassen der Streams mit primitiven Werten (z.B. `java.util.stream.IntStream`) gibt es äquivalente Methoden.
- Mit `of` werden die übergebenen Wert in einen Stream gepackt.
- Mit `iterate` und `generate` hat man eine einfache Möglichkeit unendliche Ströme zu erzeugen.

Beispiel

```
1 // Object-Stream 1, 2 ... 9, 0:
2 Stream<Integer> is1a = Stream.of(1,2,3,4,5,6,7,8,9,0);

3 // int-Stream 1, 2, ... 9, 0
4 IntStream is1b = IntStream.of(1,2,3,4,5,6,7,8,9,0);

5 // (infinite) Stream 1, 2, ...
6 Stream<Integer> is2 = Stream.iterate(1, ((x) -> x+1));

7 int[] z = new int[] {1};
8 Stream<Integer> is3 = Stream.generate(() -> z[0]++); // (infinite) Stream 1, 2, ...
```

Statische range-Methoden in `IntStream` und `LongStream`

Die Interfaces `java.util.stream.IntStream` und `java.util.stream.LongStream` enthalten jeweils zwei statische `range`-Methoden mit denen Streams erzeugt werden können.

Beispiel

```
1 IntStream isPrimA = IntStream.range(1, 10); // 1,2, .. 9
2 IntStream isPrimA = IntStream.rangeClosed(1, 10); // 1,2, .. 9, 10
```

Nicht-statische Methoden der Collection-API

Das Interface `java.util.Collection` enthält die Methode `stream` mit der die jeweilige Kollektion in einen Stream umgewandelt werden kann.

Beispiel

```
1 Stream<Integer> is = Arrays.asList(1,2,3,4,5,6,7,8,9,0).stream();
```

Verwendung von Streams

Streams werden typischerweise in einer Pipeline-artigen Struktur genutzt:

1. Erzeugung
2. Folge von Verarbeitungs-/Transformationsschritten
3. Abschluss mit einer terminalen Operation

Verarbeitungsoperationen

Verarbeitungs-Operationen transformieren die Elemente eines Streams. Man unterscheidet:

■ zustandslose Operationen

Transformieren die Elemente jeweils völlig unabhängig von allen anderen.

■ zustandsbehaftete Operationen

Transformieren die Elemente abhängig von anderen.

Zustandslose Verarbeitungsoperationen

- `map(Function<? super T, ? extends R> mapper)`: Transformiert jedes Element in ein anderes.
- `filter(Predicate<? super T> predicate)`: Filtert Elemente heraus.
- `flatMap(Function<? super T, ? extends Stream<? extends R>> mapper)`: Transformiert jedes Element in einen Stream und fügt die Streams zusammen.

Beispiel

```
1 import java.util.List;
2 import java.util.stream.Collectors;
3 import java.util.stream.IntStream;
4
5 List<Integer> is = IntStream.range(1, 10)
6     .filter(i -> i % 2 != 0)
7     .peek(i -> System.out.print(i + " "))
8     .map(i -> 10 * i)
9     .boxed()
10    .collect(Collectors.toList());
11 System.out.println(is);
```

Ausgabe: 1 3 5 7 9 [10, 30, 50, 70, 90]

Beispiel

```
1 import java.util.List;
2 import java.util.stream.Collectors;
3 import java.util.stream.IntStream;
4 import java.util.stream.Stream;
5
6 static Stream<Integer> range(int from, int to) {
7     return IntStream.range(from, to).boxed();
8 }
9
10 List<Integer> is = Stream.of(0, 1, 2)
```

```
11 .flatMap(i → range(10 * i, 10 * i + 10))
12 .collect(Collectors.toList());
```

Ausgabe: is $\Rightarrow$ [0, 1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12, 13, 14, 15, 16, 17, 18, 19, 20, 21, 22, 23, 24, 25, 26, 27, 28, 29]

Zustandsbehaftete Verarbeitungsoperationen

- `distinct()`: Entfernt Duplikate.
- `sorted()`: Sortiert die Elemente.
- `sorted(Comparator<? super T> comparator)`: Sortiert die Elemente mit einem gegebenen Comparator.
- `limit(long maxSize)`: Begrenzt die Anzahl der Elemente.
- `skip(long n)`: Überspringt die ersten n Elemente.

Beispiel

```
1 import java.util.List;
2 import java.util.stream.Collectors;
3 import java.util.stream.Stream;
4
5 List<Integer> lst = Stream.of(9, 0, 3, 1, 7, 3, 4, 7, 2, 8, 5, 0, 6, 2)
6     .distinct()
7     .sorted((i, j) → i - j)
8     .skip(1)
9     .limit(3)
10    .collect(Collectors.toList());
```

Ausgabe: is $\Rightarrow$ [1, 2, 3]

Verarbeitungsoperationen

Eine terminale Operation hat im Gegensatz zu den Verarbeitungsoperationen *keinen* *Stream* als Ergebnis.

Terminale Operationen ohne Ergebnis

- `forEach(Consumer<? super T> action)`
Wendet die übergebene Aktion auf alle Elemente des Streams an.

Terminale Operationen mit Ergebnis

- Operationen mit Array-Ergebnis: `Stream  $\Rightarrow$  Array`
Operationen die den Stream in ein äquivalentes Array umwandeln.
- Operationen mit Kollektions-Ergebnis: `Stream  $\Rightarrow$  Kollektion`
Operationen die den Stream in eine äquivalente Kollektion umwandeln.
- Operationen mit Einzel-Ergebnis: Aggregierende Operationen
Operationen die den Stream zu einem einzigen Wert verarbeiten.

Beispiel

`forEach`

```

1 Stream.of(9, 0, 3, 1, 7, 3, 4, 7, 2, 8, 5, 0, 6, 2)
2   .distinct()
3   .sorted( (i,j) → i-j )
4   .limit(3)
5   .forEach( System.out::println );

```

Beispiel

toArray

```

1 int[] a = IntStream.range(1, 3).toArray();
2
3 Object[] a = Stream.of("1", "2", "3").map( Integer::parseInt )
4   .toArray();
5
6 Integer[] a = (Integer[]) Stream.of(1, 2, 3)
7   .toArray();
8
9 String[] a = Stream.of(1, 2, 3).map( (i) → i.toString() )
10  .toArray( String[]::new ); // using generator

```

Terminale Operationen mit Kollektions-Ergebnis

- Die Methode `collect` erzeugt eine Kollektion aus den Elementen des Streams.
- `IntStream` und andere Streams mit primitiven Daten haben keine entsprechende Operation.
- Das Argument von `collect` ist ein `java.util.stream.Collector`. Die Erzeugung einer Kollektion ist damit Sonderfall einer aggregierenden Operation.
- Für die Erzeugung einer Kollektion verwendet man typischerweise einen vordefinierten `Collector` aus der Klasse `java.util.stream.Collectors`.
- Einfache Kollektionserzeuger in Collectors sind:

```

■ toList()
■ toSet()
■ toCollection(Supplier<C> collectionFactory)

```

Beispiel

collect

```

1 List<Integer> l1 = Stream.of(1, 2, 3).collect( Collectors.toList() );
2
3 List<Integer> l2 = IntStream.range(1, 4).boxed()
4   .collect( Collectors.toList() );
5
6 Set<String> s1 = (Set<String>) Stream.of("1", "2", "3")
7   .collect( Collectors.toSet());
8
9 Set<String> s2 = (Set<String>) Stream.of("1", "2", "3")
10  .collect( Collectors.toCollection( HashSet::new ) );
1
2 // Generating a map from a stream of strings

```

```

3 Map<String, Integer> m = Stream.of("1", "2", "3")
4     .collect(
5         Collectors.toMap(
6             (s) -> s,
7             Integer::parseInt
8         )
9     );

```

In `Collectors` finden sich **Kollektoren mit denen Maps erzeugt werden können**, die eine Gruppierung bzw. eine Partitionierung der Stream-Elemente darstellen:

- `static <T,K> Collector<T,?,Map<K,List<T>>> groupingBy(Function<? super T,? extends K> classifier)`

Gruppiert die Elemente entsprechend einer Klassifizierungsfunktion.

- `static <T> Collector<T,?,Map<Boolean,List<T>>> partitioningBy(Predicate<? super T> predicate)`

Partitioniert die Elemente entsprechend einem Prädikat.

Beispiel

`collect(groupingBy)`

Hilfreiche Methoden

```

1 import static java.util.stream.Collectors.groupingBy;
2 import static java.util.stream.Collectors.partitioningBy;
3 import static java.util.stream.Collectors.counting;

```

```

1 Map<Integer, List<Integer>> groupedByMod3 = Stream.of(1, 2, 3, 4, 5, 6, 7, 8, 9)
2     .collect( groupingBy( (x) -> x%3 ) );

```

Ausgabe: `groupedByMod3 = {0=[3, 6, 9], 1=[1, 4, 7], 2=[2, 5, 8]}`

```

1 Map<Integer, List<String>> groupedByLength = Stream.of(
2     "one", "two", "three", "four", "five", "six", "seven", "eight")
3     .collect( groupingBy( (s) -> s.length() ) );

```

Ausgabe: `groupedByLength ==> {3=[one, two, six], 4=[four, five], 5=[three, seven, eight]}`

Das Interface `Stream` bzw. die Interfaces für Ströme primitiver Daten (`IntStream`, etc.) bieten einige **einfache aggregierende Funktionen für Standardoperationen** auf allen Elementen des Stroms.

Beispiel

```

1 long count = Stream.of(1, 2, 3, 4, 5, 6, 7, 8, 9).count();
2
3 long sum = IntStream.of(1, 2, 3, 4, 5, 6, 7, 8, 9).sum();
4
5 OptionalDouble av = IntStream.of(1, 2, 3, 4, 5, 6, 7, 8, 9).average();

```

Das Interface `Stream` bieten einige einfache **aggregierende Funktionen für den Test aller Elemente des Stroms** mit einem übergebenen Prädikat.

Beispiel

```
1 boolean anyEven = Stream.of(1, 2, 3, 4, 5, 6, 7, 8, 9)
2   .anyMatch( (x) → x%2 == 0 );
3
4 boolean allEven = Stream.of(1, 2, 3, 4, 5, 6, 7, 8, 9)
5   .allMatch( (x) → x%2 == 0 );
6
7 boolean noneEven = Stream.of(1, 2, 3, 4, 5, 6, 7, 8, 9)
8   .noneMatch( (x) → x%2 == 0 );
```

Das Interface `Stream` bietet die Funktionen `findFirst` und `findAny` für die „Suche“ nach dem ersten bzw. irgendeinem Element in einem Stream.

⚠ Achtung!

Diese Methoden haben kein Prädikat als Parameter. Es empfiehlt sich darum den `Stream` vorher mit dem entsprechenden Prädikat zu filtern.

Beispiel

```
1 Optional<Integer> firstEven = Stream.of(1, 2, 3, 4, 5, 6, 7, 8, 9)
2   .filter( (x) → x%2 == 0 )
3   .findFirst();
```

Ausgabe: `firstEven` $\Rightarrow$ `Optional[2]`

Das Interface `Stream` bietet die Funktion

```
Optional<T> reduce(BinaryOperator<T> accumulator)
```

mit der eine Funktion auf jedes Element und das bisherige Zwischenergebnis angewendet werden kann.

Falls der erste Wert nicht der Startwert sein soll, verwendet man:

```
Optional<T> reduce(T identity, BinaryOperator<T> accumulator)
```

Beispiel

reduce

```
1 Optional<Integer> sumOfAll = Stream.of(1, 2, 3, 4, 5).reduce( (a, x) → a+x );
```

Ausgabe: `sumOfAll` $\Rightarrow$ `Optional[15]`

```
1 Optional<Integer> subOfAll = Stream.of(1, 2, 3, 4, 5).reduce( (a, x) → a-x );
```

Ausgabe: `subOfAll` $\Rightarrow$ `Optional[-13]`

```
1 int sumOfAllPlus100 = Stream.of(1, 2, 3, 4, 5)
2   .reduce(100, (a, x) → a+x );
```

Ausgabe: `sumOfAllPlus100` $\Rightarrow$ `115`

Es gibt einen Kollektor mit dem String-Elemente zu einem String konkateniert werden können:

```
static Collector<CharSequence,?,String> joining(CharSequence delimiter)
```

Beispiel

reduce

```
1 String concat = Stream.of("one", "two", "three")  
2   .collect( joining("+") );
```

Ausgabe: concat = one+two+three

Streams - fortgeschrittene Konzepte

Ausgewählte Eigenschaften des *Basisinterface* aller Streams

Parallele und sequentielle Streams.

```
1 package java.util.stream;
2
3 public interface BaseStream<T, S extends BaseStream<T,S>> {
4     /** Closes the stream, releasing any resources associated with it. */
5     void close();
6
7     /** Returns an equivalent stream that is parallel. */
8     S parallel();
9     /** Returns an equivalent stream that is sequential. */
10    S sequential();
11 }
12
13 public interface Stream<T> extends BaseStream<T, Stream<T>> {
14     // ...
15 }
```

Die Interfacedefinition (`BaseStream<T, S extends BaseStream<T,S>>`) ist eine Anwendung des CRTP; d. h. des *Curiously Recurring Template Patterns*. Bei diesem Idiom haben wir eine Klasse `X`, die von einer generischen Klasse oder einem generischen Interface `S` abgeleitet wird, wobei die ableitende Klasse `X` sich selber als Typparameter verwendet. Dies erlaubt die Definition einer Fluent-API, bei der Methoden, die in der Basisklasse definiert sind, den abgeleiteten Typ zurückgeben.

Erzeugen von eigenen Streams mittels `StreamSupport`

Die Implementierung des Interfaces `Stream<T>` ist ggf. sehr aufwändig. Alternativ kann die Klasse `StreamSupport` verwendet werden, um auf einem `Splititerator` basierende Streams zu erzeugen.

```
1 package java.util.stream;
2
3 public final class StreamSupport {
4
5     /** Creates a new sequential or parallel Stream from a Splititerator. */
6     static <T> Stream<T> stream(Splititerator<T> spliterator, boolean parallel);
7
8     // ...
9 }
```


Java Optionals

Instanzen der Klasse `java.util.Optional<T>` (bzw. `java.util.OptionalInt` etc.) **repräsentieren Werte die vorhanden sind oder auch nicht**.

Insbesondere `java.util.Optional<T>` kann/sollte anstelle von `null` verwendet werden, in Fällen in denen unter bestimmten Umständen kein sinnvoller Wert angegeben werden kann.

◆ Bemerkung

Es gibt moderne Programmiersprachen, die auf `null` komplett verzichten und stattdessen immer auf `Optionals` oder ähnliche Konstrukte setzen.

📄 Beispiel

```
1 static Optional<Integer> min(int[] a ) {  
2     if(a == null || a.length == 0)  
3         return Optional.empty();  
4  
5     int min = a[0];  
6     for(int x: a) { if (x < min) { min = x; } }  
7     return Optional.of(min);  
8 }
```

Übung - Streams Grundlagen

0.1. Streams und Lambda Ausdrücke

```
1 void main() {
2     List<String> names = Arrays.asList("Alice", "Bob", "Charlie", "Diana", "Eve");
3     List<Integer> grades = Arrays.asList(85, 42, 91, 67, 55);
4
5     // TODO 1: Using a lambda, filter the grades list to only keep grades ≥ 60
6     // and print the passing grades.
7     // Hint: Use stream(), filter(), and forEach().
8
9
10    // TODO 2: Create a Predicate<Integer> lambda called `isExcellent`
11    // that returns true if a grade is 90 or above. Then print all excellent grades.
12
13
14    // TODO 3: Create a Function<Integer, String> lambda called `toLetterGrade`
15    // that converts a numeric grade to a letter:
16    // 90+ → "A"; 75+ → "B" ; 60+ → "C"; below 60 → "F"
17    // Then print each grade alongside its letter grade.
18
19
20    // TODO 4: Using a lambda, calculate and print the average of all grades.
21    // Hint: Use stream() and mapToInt().
22 }
```

Übung

0.2. Java Streams

Bemerkung

Verwenden Sie ausschließlich Streams und Lambda-Ausdrücke.

1. Schreiben Sie eine Methode `int sumOfSquares(int[] a)` die die Elemente des Arrays quadriert und dann die Summe berechnet.
2. Schreiben Sie eine Methode `int sumOfSquaresEven(int[] a)` die die Elemente des Arrays quadriert, und dann die Summe berechnet für alle Elemente die gerade sind.
3. Schreiben Sie eine Methode, die eine Liste von Strings (`List<String>`) in eine flache Liste von Zeichen (`List<Integer>`) umwandelt.
4. Schreiben Sie eine Methode, die die Zahlen von 1 bis `Integer.MAX_VALUE` addiert. Nutzen Sie `IntStream.range()` um die Zahlen zu iterieren. Messen Sie die Ausführungsdauer für die *sequentielle* und *parallele* Ausführung (siehe Anhang für eine entsprechende Methode zur Zeitmessung.)

Um die Ausführungsdauer Ihrer Methode zu messen, können Sie folgenden Methode verwenden:

```
1 void time(Runnable r) {  
2     final var startTime = System.nanoTime();  
3     r.run();  
4     final var endTime = System.nanoTime();  
5     System.out.println("elapsed time: " + (endTime - startTime));  
6 }
```

Ein Aufruf der Methode `time` könnte dann so aussehen:

```
1 time(() -> System.out.println(sumOfSquares(new int[]{1,2,3,4,5,6,7,8,9,0})));
```

Herausforderungen gelöst?

- Streams unterstützen die korrekte Verarbeitung von (Massen-)Daten durch die Anwendung von funktionalen und deklarativen Konzepten mit dem Ziel einer möglichst geringe Repräsentationslücke (📊 *Low representational gap*).

Vergleich von Java's Stream-API mit Scala's Stream-API